

LAPORAN TUGAS BESAR
IF2224 TEORI BAHASA FORMAL DAN OTOMATA

MILESTONE 2



Disusun oleh:

Gabriella Botimada Lubis - 13524006

Stefani Angeline Oroh - 13524064

Reva Natania Sitohang - 13524098

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
BANDUNG

2026

DAFTAR ISI

DAFTAR ISI.....	3
DAFTAR TABEL.....	4
DAFTAR GAMBAR.....	5
BAB I	
DASAR TEORI.....	6
1.1 Bahasa Pemrograman dan Proses Kompilasi.....	6
1.2 Lexical Analysis.....	6
1.2.1 Fungsi dan Peran Lexer dalam Proses Kompilasi.....	6
1.2.2 Token.....	7
1.2.3 Cara Kerja Lexer.....	8
1.3 Deterministic Finite Automata (DFA).....	8
BAB II	
PERANCANGAN DAN IMPLEMENTASI.....	9
2.1 Diagram DFA.....	10
2.1.1 Daftar Token Bahasa Pemrograman Airon.....	11
2.2 Implementasi Program.....	18
2.2.1 Perancangan Program.....	18
2.3 Implementasi Kode.....	22
2.3.1 Implementasi Token.....	22
2.3.2 Implementasi DFA.....	22
2.3.3 Implementasi Lexer.....	25
BAB III	
PENGUJIAN.....	27
3.1 Test Case 1.....	27
3.2 Test Case 2.....	28
3.3 Test Case 3.....	30
3.4 Test Case 4.....	34
3.5 Test Case 5.....	37
3.6 Test Case 6.....	42
3.7 Test Case 7.....	43
BAB IV	
KESIMPULAN DAN SARAN.....	46
4.1 Kesimpulan.....	46
4.2 Saran.....	46
LAMPIRAN.....	48

DAFTAR TABEL

Tabel 3.1 Input/Output Test Case 1.....	13
Tabel 3.2 Input/Output Test Case 2.....	15
Tabel 3.3 Input/Output Test Case 3.....	18
Tabel 3.4 Input/Output Test Case 4.....	22
Tabel 3.5 Input/Output Test Case 5.....	26
Tabel 3.6 Input/Output Test Case 6.....	30
Tabel 3.7 Input/Output Test Case 7.....	31
Tabel 3.8 Input/Output Test Case 8.....	31
Tabel 3.9 Input/Output Test Case 9.....	31
Tabel 3.10 Input/Output Test Case 10.....	32
Tabel 3.11 Input/Output Test Case 11.....	32
Tabel 3.12 Input/Output Test Case 12.....	33

DAFTAR GAMBAR

Gambar 2.1 (1) Diagram DFA.....	10
Gambar 2.1 (2) Lanjutan Diagram DFA.....	10
Gambar 2.3.1 Implementasi Token.....	21
Gambar 2.3.2 (1) State DFA pada Kode.....	22
Gambar 2.3.2 (2) Implementasi Diagram Transisi DFA.....	24
Gambar 2.3.3 Implementasi Lexer.....	24

BAB I

DASAR TEORI

1.1 Bahasa Pemrograman dan Proses Kompilasi

Bahasa pemrograman adalah sistem notasi formal yang digunakan untuk mengekspresikan algoritma dan instruksi yang dapat dieksekusi oleh komputer. Karena CPU hanya memahami bahasa mesin (binary 0 dan 1), diperlukan suatu mekanisme untuk menerjemahkan kode sumber yang ditulis manusia menjadi kode mesin. Terdapat dua pendekatan utama dalam proses ini, yaitu *compiler* dan *interpreter*. *Compiler* menerjemahkan seluruh kode sumber menjadi kode mesin sebelum program dieksekusi, menghasilkan file objek yang siap dijalankan. Sementara itu, *interpreter* memproses dan mengeksekusi kode sumber secara langsung baris per baris tanpa menghasilkan file objek terlebih dahulu. Meskipun cara kerjanya berbeda, kedua pendekatan ini pada dasarnya melewati tahapan analisis yang serupa, yaitu *Lexical Analysis*, *Syntax Analysis*, *Semantic Analysis*, dan *Intermediate Code Generation*.

1.2 Syntax Analysis (Parser)

Syntax Analysis atau parsing adalah proses memeriksa apakah deretan token membentuk struktur sintaksis yang valid sesuai grammar bahasa pemrograman. Komponen yang melakukan proses ini disebut parser.

Parser memiliki tiga fungsi utama:

- Verifikasi Sintaksis

Memastikan urutan token sesuai dengan grammar, mendeteksi dan melaporkan syntax error dengan pesan yang informatif mencakup nomor baris, kolom, token yang ditemukan, dan token yang diharapkan.

- Pembangunan Struktur Hierarkis

Mengorganisasikan token-token datar menjadi struktur pohon yang mencerminkan hierarki gramatikal program.

- Persiapan Tahap Berikutnya

Menghasilkan Parse Tree yang menjadi masukan bagi tahap Semantic Analysis.

1.3 Grammar dan *Context Free Grammar* (CFG)

Grammar bahasa pemrograman didefinisikan sebagai Context-Free Grammar (CFG), yaitu sekumpulan aturan produksi yang mendefinisikan struktur sintaksis bahasa. Sebuah CFG terdiri dari empat komponen:

- $V \rightarrow$ himpunan simbol non-terminal. Contoh: $\langle \text{expression} \rangle$, $\langle \text{statement} \rangle$
- $\Sigma \rightarrow$ himpunan simbol terminal (token), contoh: ident, plus, intcon
- $P \rightarrow$ himpunan aturan produksi berbentuk $A \rightarrow \alpha$, di mana $A \in V$ dan $\alpha \in (V \cup \Sigma)^*$
- $S \rightarrow$ simbol awal (start symbol), pada Arion adalah $\langle \text{program} \rangle$

Setiap aturan produksi $A \rightarrow \alpha$ berarti non-terminal A dapat "diturunkan" menjadi string α . Proses menggantikan non-terminal secara berurutan dari simbol awal hingga menghasilkan seluruh token disebut derivasi. Bahasa Arion menggunakan leftmost derivation, yaitu non-terminal paling kiri selalu dikembangkan terlebih dahulu — dan inilah yang dilakukan Recursive Descent.

1.4 Parse Tree

Parse Tree (atau Concrete Syntax Tree) adalah representasi grafis dari derivasi suatu string dalam sebuah grammar. Setiap node merepresentasikan satu simbol grammar:

- Node akar: simbol awal $\langle \text{program} \rangle$, selalu menjadi root.

- Node internal: simbol non-terminal seperti `<expression>`, `<term>`. Memiliki anak sesuai ruas kanan aturan produksi yang diterapkan.
- Node daun (leaf): simbol terminal (token) seperti `ident(a)`, `plus`, `intcon(5)`. Tidak memiliki anak.
- Node epsilon (ϵ): non-terminal tanpa anak, valid untuk aturan produksi yang mengizinkan string kosong, contohnya `<statement>` kosong setelah semicolon terakhir.

Parse Tree berbeda dengan AST (Abstract Syntax Tree). Parse Tree memuat seluruh simbol terminal dan non-terminal termasuk elemen gramatikal seperti tanda kurung, titik koma, dan kata kunci. AST hanya memuat informasi semantik yang relevan dan menghilangkan detail tersebut. Pada Milestone 2 ini, yang dibangun adalah Parse Tree.

Sifat penting Parse Tree adalah mencerminkan prioritas operator secara implisit melalui kedalaman pohon. Pada ekspresi $2 + 3 * 4$, node perkalian berada lebih dalam (di `<term>`) dibanding penjumlahan (di `<simple-expression>`), sehingga perkalian otomatis memiliki prioritas lebih tinggi tanpa logika khusus

1.5 Recursive Descent Parsing

Recursive Descent Parser adalah teknik parsing top-down di mana setiap aturan produksi grammar dipetakan secara langsung menjadi satu fungsi rekursif dalam kode program. Parser memulai dari simbol awal `<program>` dan secara rekursif menurunkan struktur program sesuai token yang dibaca satu per satu.

1.5.1 Prinsip Kerja

Setiap fungsi parse bekerja dengan cara berikut :

1. Baca token saat ini melalui `current()` tanpa look ahead
2. Berdasarkan token yang terlihat, memilih aturan produksi yang sesuai

3. Untuk setiap simbol di ruas kanan aturan, jika terminal maka panggil expect (type), dan jika non-terminal maka panggil parseYyy() secara rekursif.
4. Setiap elemen yang berhasil di look ahead atau dihasilkan ditambahkan sebagai *child node*
5. Jika token tidak sesuai, lempar SyntaxError dengan pesan yang sesuai

1.5.2 Look-ahead dan Disambiguasi

Recursive descent menggunakan look-ahead, yaitu kemampuan untuk melihat satu atau beberapa token ke depan tanpa mengonsumsinya, untuk memutuskan aturan mana yang akan diterapkan. Ini diperlukan ketika beberapa aturan dimulai dengan token yang sama seperti :

Konteks	Token Saat Ini	Look-ahead (peek(1))	Keputusan
parseStatement()	ident	becomes	parseAssignmentStatement()
parseStatement()	ident	lparent	parseProcFuncCall()
parseStatement()	ident	lain-lain	parseProcFuncCall() tanpa argumen
parseFactor()	ident	lparent	parseProcFuncCall()
parseFactor()	ident	lain-lain	parseVariable()
parseType()	ident	period	parseRange()
parseType()	ident	lain-lain	ident tunggal (tipe bernama)

1.5.3 Kelebihan dan Keterbatasan

Kelebihan recursive descent adalah mudah diimplementasikan karena korespondensi langsung grammar ↔ kode memudahkan pemahaman dan pemeliharaan, error recovery karena posisi dalam call stack menunjukkan

konteks saat ini sehingga pesan error dapat sangat informatif, dan fleksibel karena recursive descent mudah menangani konstruk khusus dengan look-ahead tambahan.

Keterbatasan utamanya adalah tidak dapat menangani left recursion secara langsung. Aturan seperti $A \rightarrow A \alpha \mid \beta$ akan menyebabkan infinite recursion. Grammar Arion telah dirancang tanpa left recursion sehingga Recursive Descent dapat diterapkan langsung.

BAB II

PERANCANGAN DAN IMPLEMENTASI

2.1 Teknologi yang Digunakan

Implementasi parser menggunakan bahasa pemrograman C++. Pilihan ini sejalan dengan implementasi lexer pada Milestone 1 sehingga integrasi antara dua komponen berlangsung dengan baik dengan antarmuka yang sudah ada.

2.1.1 Bahasa Pemrograman - C++

C++ dipilih karena beberapa alasan teknis yang sesuai dengan kebutuhan membangun sebuah compiler:

- Manajemen memori eksplisit dengan bantuan smart pointer (`std::shared_ptr`) memungkinkan pembangunan struktur pohon (parse tree) yang bersifat rekursif tanpa risiko memory leak.
- Fitur `std::initializer_list` memungkinkan penulisan `checkAny({...})` yang ringkas dan mudah dibaca saat memeriksa beberapa jenis token sekaligus.
- Performa tinggi sangat relevan karena proses parsing melibatkan iterasi berulang atas deretan token.
- Ekosistem standar library (STL) yang kaya: `std::vector` untuk menyimpan token, `std::ostringstream` untuk memformat pesan error, dan `std::ostream` sebagai antarmuka output yang fleksibel.

2.1.2 Manajemen Memori - `std::shared_ptr`

Setiap node parse tree direpresentasikan sebagai `shared_ptr<ParseNode>`. Penggunaan shared pointer dipilih karena:

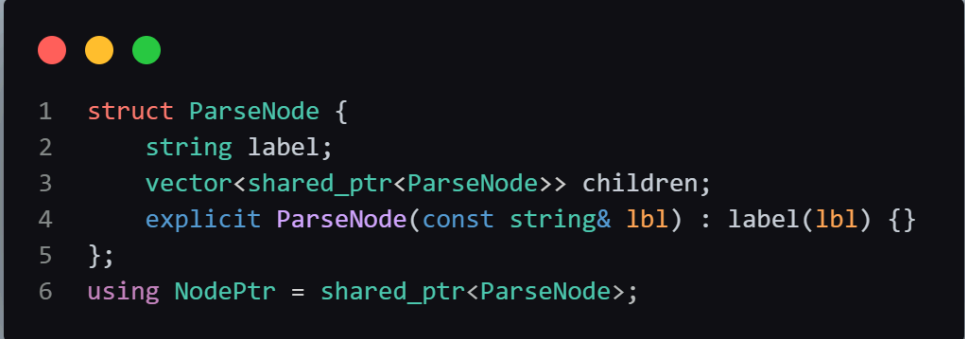
- Node parse tree bersifat hierarkis dan setiap node dapat memiliki banyak anak (children), sehingga pemilikan memori bersifat shared secara natural.
- Tidak perlu melakukan dealokasi manual. Reference counting memastikan node dihapus otomatis ketika tidak ada referensi aktif.
- Tipe alias `NodePtr = shared_ptr<ParseNode>` menjaga keterbacaan kode, terutama pada nilai kembalian setiap fungsi parse.

2.2 Struktur Program

File	Modul	Tanggung Jawab
token.hpp / token.cpp	Token	Mendefinisikan enum TokenType dan struct Token yang menjadi satuan data bersama antara lexer dan parser.
lexer.hpp / lexer.cpp	Lexer	Implementasi DFA berbasis state-machine untuk mengubah karakter sumb2Wfer menjadi stream token. Digunakan sebagai tahap pertama sebelum parser dipanggil.
parser.hpp	Parser (header)	Mendeklarasikan struct ParseNode, tipe alias NodePtr, class SyntaxError, serta seluruh method parsing milik class Parser.
parser.cpp	Parser (impl.)	Implementasi lengkap seluruh fungsi parse yang menerapkan algoritma Recursive Descent sesuai grammar bahasa Arion.
main.cpp	Driver	Titik masuk program: menerima path file, menjalankan lexer, meneruskan token ke parser, mencetak parse tree ke terminal, dan menyimpan ke file output.

2.2.1 Struktur Data Utama - ParseNode

Node parse tree direpresentasikan oleh struct ParseNode yang sederhana tapi cukup untuk merepresentasikan seluruh hierarki sintaksis

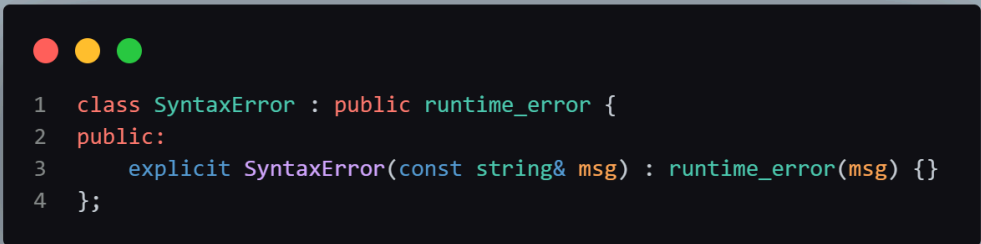


```
1 struct ParseNode {  
2     string label;  
3     vector<shared_ptr<ParseNode>> children;  
4     explicit ParseNode(const string& lbl) : label(lbl) {}  
5 };  
6 using NodePtr = shared_ptr<ParseNode>;
```

Field label diisi dengan nama non-terminal atau label terminal yang dilengkapi dengan nilai token. Field children adalah daftar urutan node anak yang merepresentasikan turunan produksi grammar

2.2.2 Error Handling - SyntaxError

Error sintaksis direpresentasikan melalui class SyntaxError yang mewarisi std::runtime_error



```
1 class SyntaxError : public runtime_error {  
2 public:  
3     explicit SyntaxError(const string& msg) : runtime_error(msg) {}  
4 };
```

Setiap kali parser menemukan token yang tidak sesuai dengan ekspektasi grammar, sebuah SyntaxError dilempar beserta pesan yang mencantumkan nomor baris, kolom,

token yang ditemukan, dan token yang diharapkan. Error ini ditangkap di main.cpp, dicetak ke terminal, dan disimpan ke file output.

2.3 Fungsi dan Kelas Utama

2.3.1 Class Parser

```
1  class Parser {
2  public:
3      explicit Parser(const vector<Token>& tokens);
4      NodePtr parse();
5      static void printTree(const NodePtr& node, ostream& out,
6                           const string& prefix = "", bool isLast = true);
7
8  private:
9      vector<Token> tokens;
10     size_t pos;
11
12     const Token& current() const;           // token saat ini
13     const Token& peek(int offset = 1) const; // look-ahead
14     bool check(TokenType t) const;
15     bool checkAny(initializer_list<TokenType> types) const;
16     NodePtr expect(TokenType t);
17     NodePtr consume();
18     NodePtr parseProgram();
19     NodePtr parseProgramHeader();
20     NodePtr parseDeclarationPart();
```

Seluruh logika parsing ada dalam class Parser. Berikut ini adalah kelompok method beserta fungsinya:

Method	Deskripsi
Parser(tokens)	Konstruktor: menerima vector<Token> hasil lexer dan menginisialisasi pos = 0.
parse()	Titik masuk parsing; memanggil parseProgram() dan memvalidasi token akhir berupa EOF.

Method	Deskripsi
<code>current()</code> / <code>peek(offset)</code>	Mengakses token pada posisi saat ini atau offset tertentu ke depan (lookahead) tanpa menggeser posisi.
<code>check(t)</code> / <code>checkAny({...})</code>	Memeriksa apakah token saat ini bertipe <code>t</code> , atau salah satu dari kumpulan tipe yang diberikan.
<code>consume()</code>	Membuat leaf node dari token saat ini, lalu menggeser posisi (<code>pos++</code>) ke token berikutnya.
<code>expect(t)</code>	Memanggil <code>consume()</code> jika token sesuai, atau melempar <code>SyntaxError</code> jika tidak sesuai.
<code>printTree(node, out, ...)</code>	Mencetak parse tree secara rekursif ke ostream (terminal atau file) menggunakan karakter box-drawing Unicode.
<code>isRelationalOp()</code> / <code>isAdditiveOp()</code> / <code>isMultiplicativeOp()</code>	Helper untuk memeriksa apakah token saat ini adalah operator pada golongan tertentu.
<code>isConstantStart()</code> / <code>isTypeStart()</code>	Helper untuk memeriksa apakah token saat ini dapat mengawali produksi <code><constant></code> atau <code><type></code> .

2.3.2 Kelompok Fungsi Parse

Setiap produksi grammar diimplementasikan sebagai satu fungsi terpisah. Pengelompokan fungsional adalah sebagai berikut:

a. Struktur Program Tingkat Atas

- `parseProgram()` – produksi `<program>`
- `parseProgramHeader()` – produksi `<program-header>`
- `parseDeclarationPart()` – produksi `<declaration-part>`
- `parseBlock()` – produksi `<block>` yang digunakan di dalam subprogram

b. Deklarasi

- `parseConstDeclaration()` dan `parseConstant()` – deklarasi konstanta
- `parseTypeDeclaration()` dan `parseType()` – deklarasi tipe
- `parseVarDeclaration()` – deklarasi variabel
- `parseArrayType()`, `parseRange()`, `parseEnumerated()`, `parseRecordType()`, `parseFieldList()`, `parseFieldPart()` – bentuk-bentuk tipe data

c. Subprogram

- `parseSubprogramDeclaration()` – dispatcher ke procedure atau function
- `parseProcedureDeclaration()` – deklarasi prosedur
- `parseFunctionDeclaration()` – deklarasi fungsi (termasuk tipe kembalian)
- `parseFormalParameterList()` dan `parseParameterGroup()` – daftar parameter formal

d. Statement

- `parseCompoundStatement()` – blok begin...end
- `parseStatementList()` – urutan statement dipisah semicolon
- `parseStatement()` – dispatcher ke semua jenis statement
- `parseAssignmentStatement(varNode)` – statement penugasan
- `parseIfStatement()`, `parseCaseStatement()`, `parseCaseBlock()` – statement kondisional
- `parseWhileStatement()`, `parseRepeatStatement()`, `parseForStatement()` – statement perulangan
- `parseProcFuncCall()` dan `parseParameterList()` – pemanggilan prosedur/fungsi

e. Variabel dan Ekspresi

- `parseVariable()` dan `parseComponentVariable()` – akses elemen array atau field record
- `parseIndexList()` – indeks array
- `parseExpression()` – ekspresi lengkap

- `parseSimpleExpression()` – ekspresi tanpa operator relasional
- `parseTerm()` dan `parseFactor()` – tingkat prioritas operator
- `parseRelationalOperator()`, `parseAdditiveOperator()`,
`parseMultiplicativeOperator()` – node operator

2.4 Alur Kerja Program

Program Arion Compiler bekerja dalam alur seperti berikut:

1. Input: pengguna memasukkan path file sumber (.txt) melalui stdin.
2. Lexical Analysis: objek Lexer membaca file karakter per karakter menggunakan DFA. Setiap token yang dihasilkan dikumpulkan ke dalam `std::vector<Token>`, dengan token COMMENT dilewati (tidak dimasukkan ke vector).
3. Konstruksi Parser: `vector<Token>` diserahkan ke konstruktor Parser. Parser menyimpan vector ini dan menginisialisasi `pos = 0` sebagai cursor.
4. Parsing (Recursive Descent): metode `parse()` dipanggil, yang kemudian secara rekursif memanggil fungsi-fungsi parse sesuai produksi grammar bahasa Arion.
5. Pembangunan Parse Tree: setiap fungsi parse membuat objek `ParseNode` dengan label non-terminal yang sesuai, kemudian menambahkan node-node anak sebagai hasil dari sub-produksi grammar.
6. Output ke Terminal: parse tree yang sudah terbentuk dicetak ke stdout menggunakan `Parser::printTree()` dengan format indentasi berbasis karakter box-drawing Unicode (`|`, `└─`, `├─`, `|`).
7. Output ke File: parse tree yang sama juga ditulis ke file `test/milestone-2/output-<nama-file>.txt`.
8. Penanganan Error: jika terjadi `SyntaxError` di langkah 4, pesan error dicetak ke terminal dan disimpan ke file output, lalu program keluar dengan kode 1.

2.4.1 Alur Lookahead dan disambiguation

Parser menggunakan lookahead 1 token ($k=1$) melalui fungsi `peek(1)` untuk menyelesaikan ambiguitas. Beberapa situasi yang memerlukan lookahead adalah:

- Pada `parseStatement()`

Jika token saat ini adalah IDENT, parser melihat satu token ke depan. Jika token berikutnya adalah LPARENT, alur diarahkan ke `parseProcFuncCall()`. Jika bukan, parser membaca variabel terlebih dahulu via `parseVariable()` kemudian memutuskan antara assignment (BECOMES) atau procedure-call tanpa argumen.

- Pada `parseType()`

Jika token saat ini adalah IDENT diikuti PERIOD, produksi diarahkan ke `parseRange()`. Jika tidak, IDENT dikonsumsi sebagai nama tipe sederhana.

- Pada `parseFieldList()`

Semicolon dikonsumsi hanya kalau token berikutnya adalah IDENT, untuk membedakan pemisah field dari semicolon penutup record.

- Pada `parseFormalParameterList()`

Semicolon dikonsumsi jika token berikutnya bukan RPARENT, untuk mencegah trailing semicolon sebelum penutup.

2.4.2 Penanganan ϵ -production

Produksi grammar yang mengizinkan statement kosong (ϵ) ditangani secara eksplisit. Pada `parseStatementList()`, setiap iterasi memanggil `parseStatement()` yang dapat mengembalikan node `<statement>` tanpa children jika token saat ini bukan salah satu kata kunci statement maupun IDENT. Ini memungkinkan penulisan seperti "begin ; end" atau "begin end" yang keduanya valid.

```

1  NodePtr Parser::parseStatement() {
2      auto node = std::make_shared<ParseNode>("<statement>");
3
4      if (check(BEGINSV)) {
5          node->children.push_back(parseCompoundStatement());
6
7      } else if (check(IFS)) {
8          node->children.push_back(parseIfStatement());
9
10     } else if (check(CASESV)) {
11         node->children.push_back(parseCaseStatement());
12
13     } else if (check(WHILESV)) {
14         node->children.push_back(parseWhileStatement());
15
16     } else if (check(REPEATSV)) {
17         node->children.push_back(parseRepeatStatement());
18
19     } else if (check(FORSV)) {
20         node->children.push_back(parseForStatement());
21
22     } else if (check(IDENT)) {
23         if (peek(1).type == LPARENT) {
24             node->children.push_back(parseProcFuncCall());
25
26         } else {
27             NodePtr varNode = parseVariable();
28
29             if (check(BECOMES)) {
30                 node->children.push_back(parseAssignmentStatement(varNode));
31             } else {
32
33                 auto callNode = std::make_shared<ParseNode>("<procedure/function-call>");
34                 if (!varNode->children.empty())
35                     callNode->children.push_back(varNode->children[0]);
36                 else
37                     callNode->children.push_back(varNode);
38                 node->children.push_back(callNode);
39             }
40         }
41     }
42
43     return node;
44 }

```

BAB III

PENGUJIAN

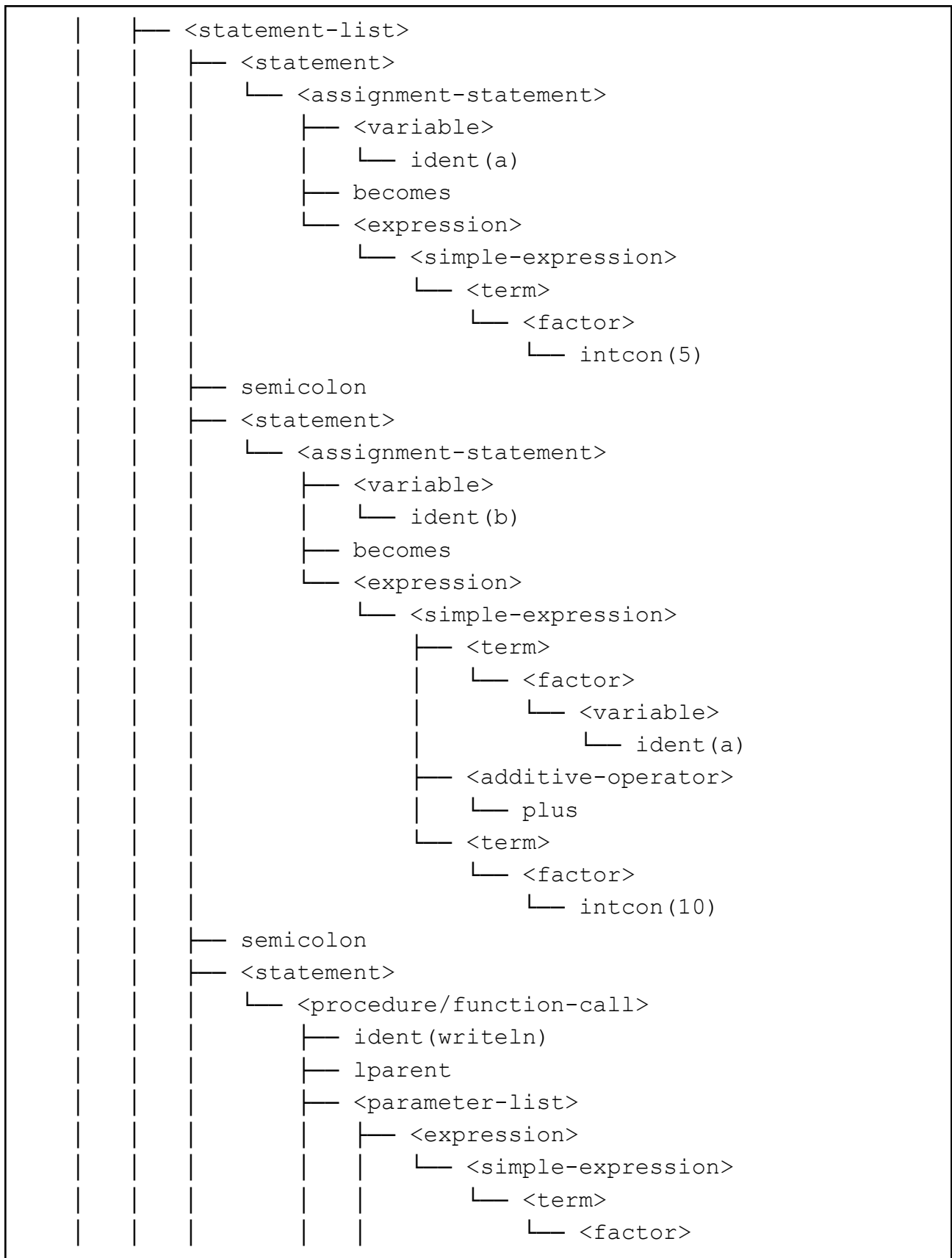
3.1 Test Case 1

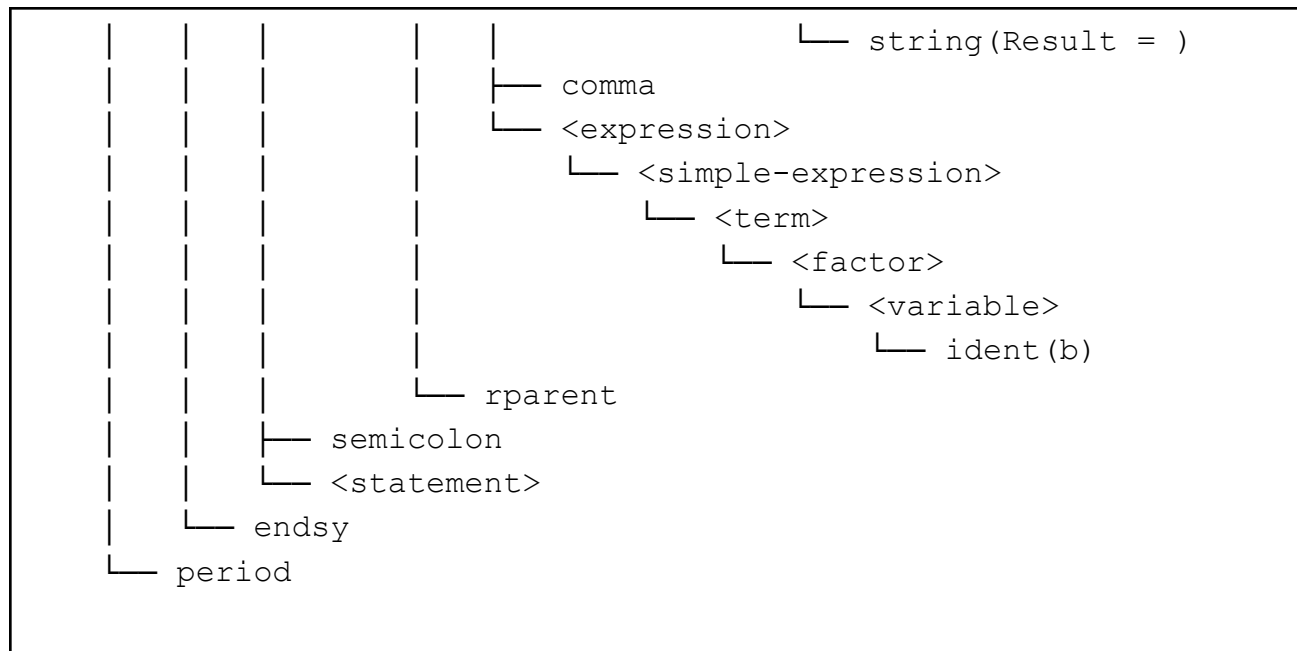
Masukan (input)

```
program Hello;  
  
var  
  a, b: integer;  
  
begin  
  a := 5;  
  b := a + 10;  
  writeln('Result = ', b);  
end.
```

Keluaran (output)

```
└─ <program>  
  └─ <program-header>  
    └─ programsy  
    └─ ident(Hello)  
      └─ semicolon  
  └─ <declaration-part>  
    └─ <var-declaration>  
      └─ varsy  
      └─ <identifier-list>  
        └─ ident(a)  
        └─ comma  
          └─ ident(b)  
      └─ colon  
      └─ <type>  
        └─ ident(integer)  
      └─ semicolon  
  └─ <compound-statement>  
    └─ beginsy
```





Tabel 3.1 Input/Output Test Case 1

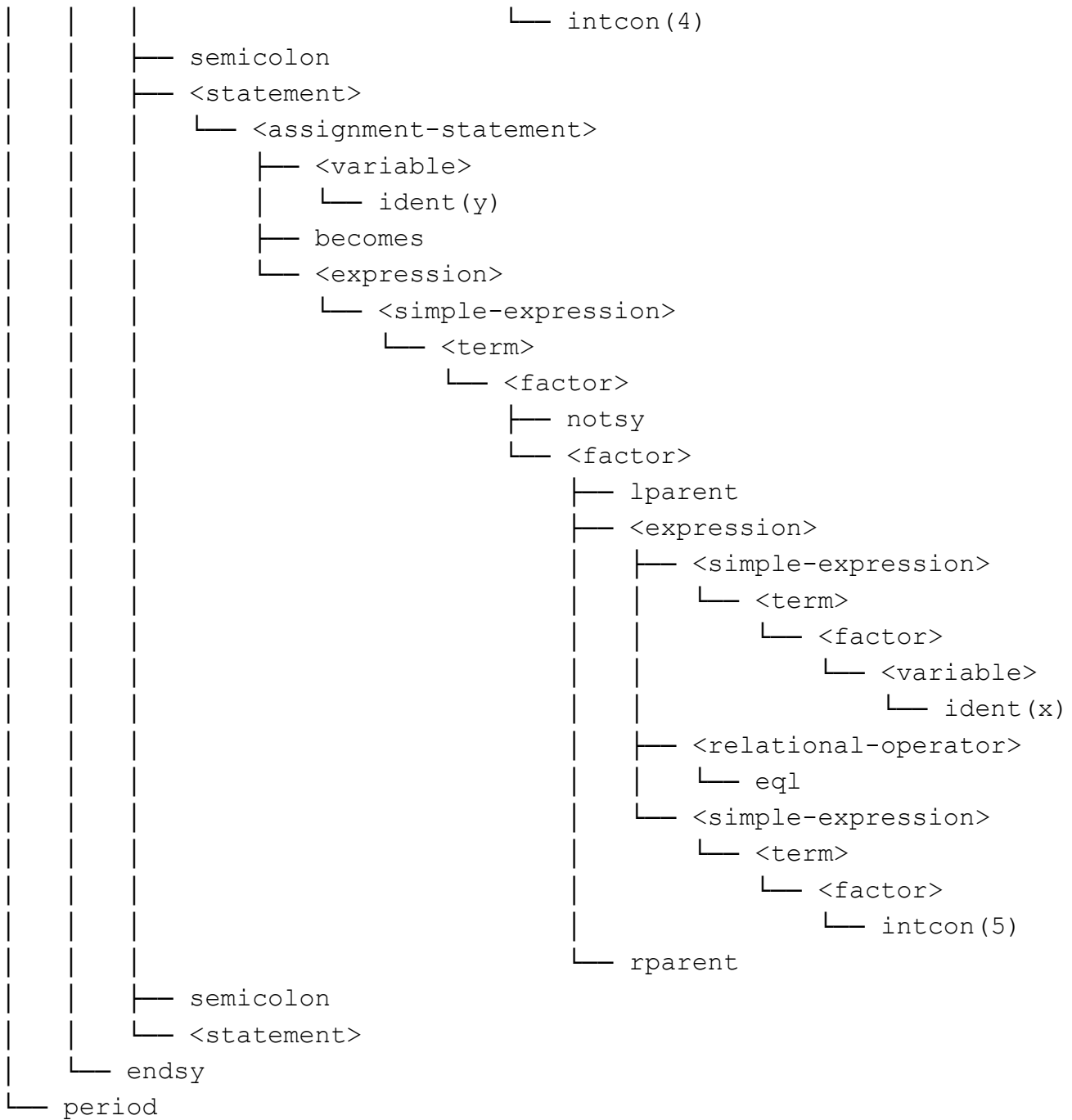
3.2 Test Case 2

Masukan (input)
<pre> program ExprTest; var x, y, z: integer; flag: boolean; begin x := 2 + 3 * 4; y := not (x == 5); end. </pre>
Keluaran (output)
<pre> └─ <program> └─ <program-header> └─ programsy └─ ident(ExprTest) └─ semicolon └─ <declaration-part> └─ <var-declaration> </pre>

```

├─ varsy
├─ <identifier-list>
│   ├── ident(x)
│   ├── comma
│   ├── ident(y)
│   ├── comma
│   └─ ident(z)
├─ colon
├─ <type>
│   └─ ident(integer)
├─ semicolon
├─ <identifier-list>
│   └─ ident(flag)
├─ colon
├─ <type>
│   └─ ident(boolean)
└─ semicolon
├─ <compound-statement>
│   ├── beginsy
│   ├── <statement-list>
│   │   ├── <statement>
│   │   │   └─ <assignment-statement>
│   │   │       ├── <variable>
│   │   │       │   └─ ident(x)
│   │   │       ├── becomes
│   │   │       └─ <expression>
│   │   │           └─ <simple-expression>
│   │   │               ├── <term>
│   │   │               │   └─ <factor>
│   │   │               │       └─ intcon(2)
│   │   │               ├── <additive-operator>
│   │   │               │   └─ plus
│   │   │               └─ <term>
│   │   │                   ├── <factor>
│   │   │                   │   └─ intcon(3)
│   │   │                   ├── <multiplicative-operator>
│   │   │                   │   └─ times
│   │   │                   └─ <factor>

```



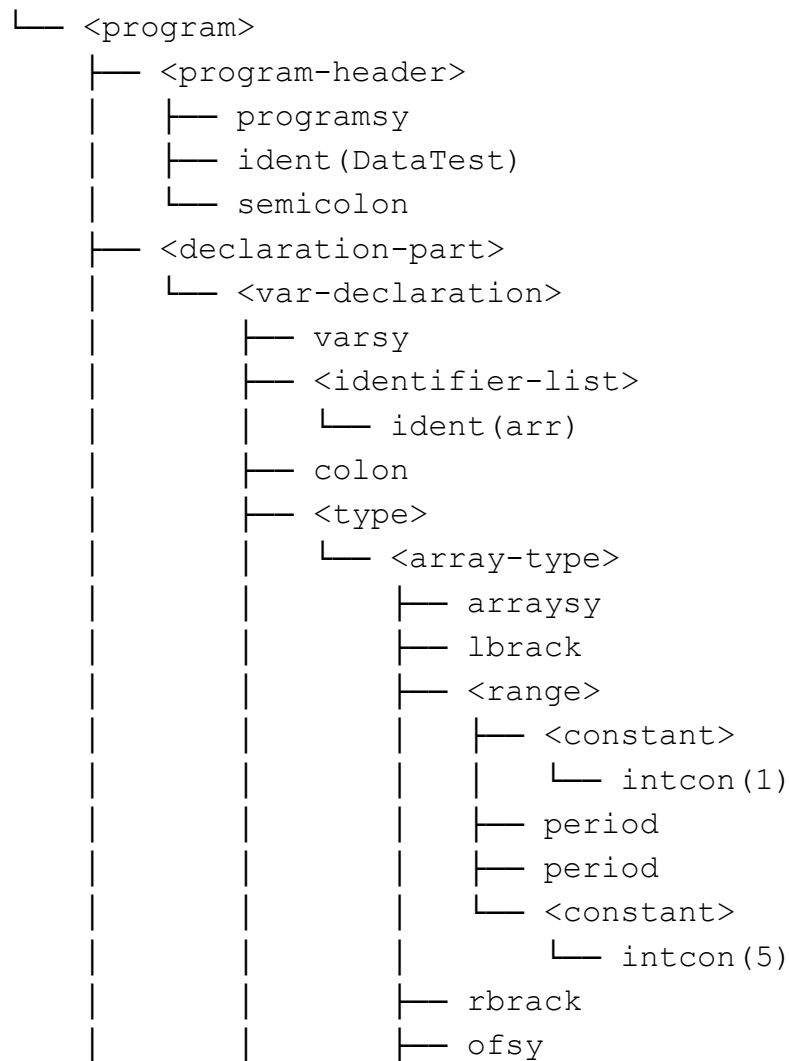
Tabel 3.2 Input/Output Test Case 2

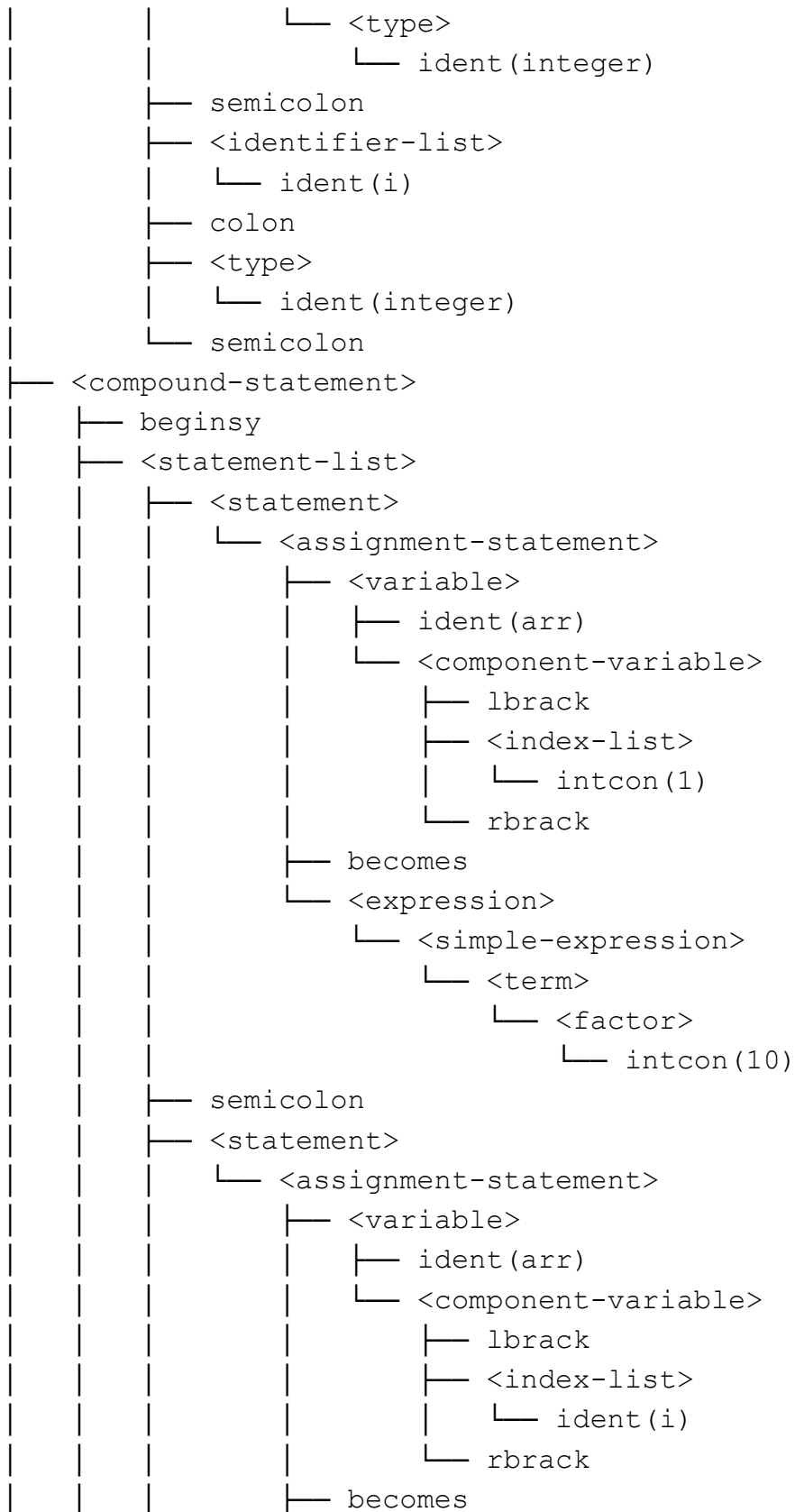
3.3 Test Case 3

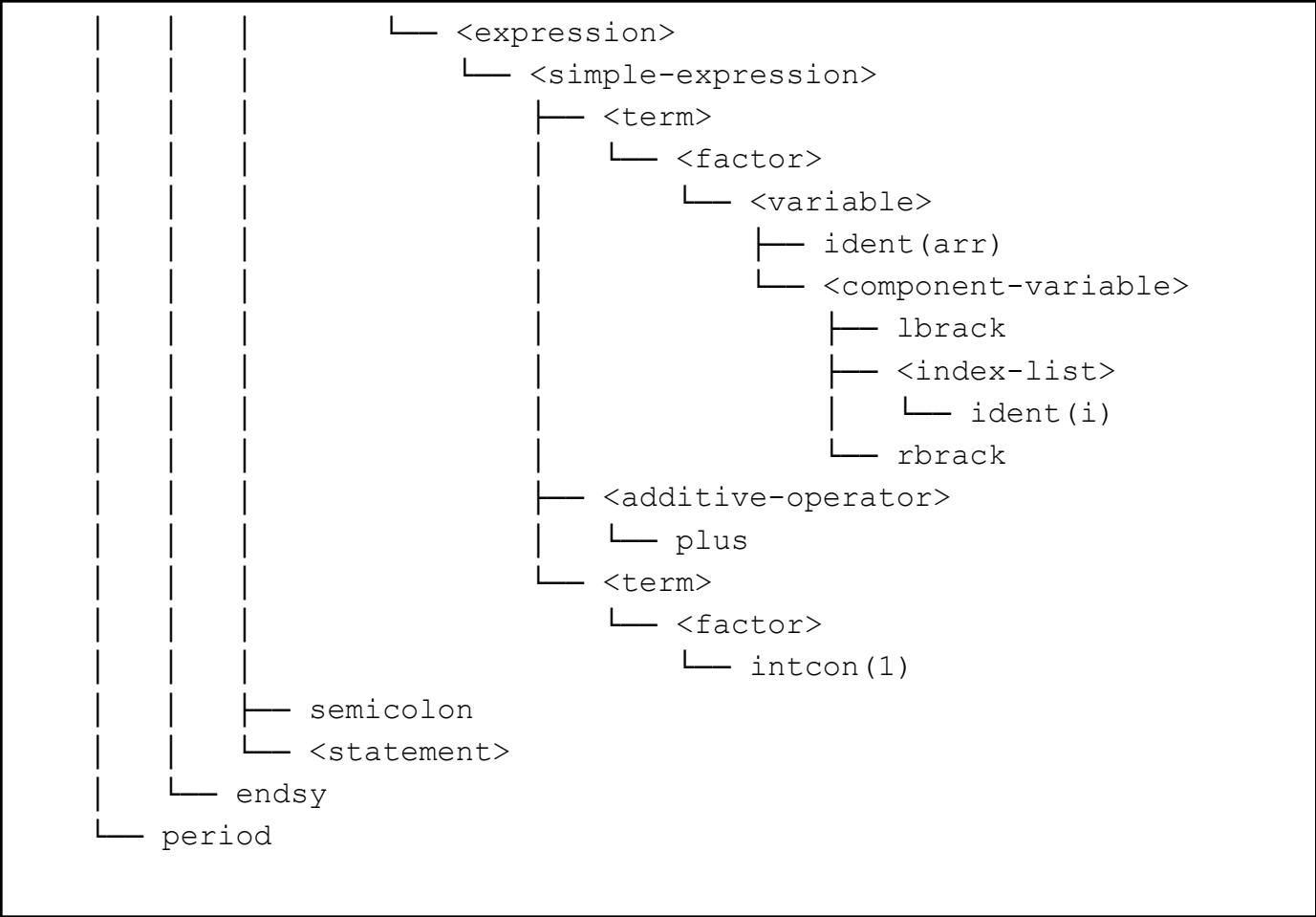
Masukan (input)

```
program DataTest;
var
  arr: array[1..5] of integer;
  i: integer;
begin
  arr[1] := 10;
  arr[i] := arr[i] + 1;
end.
```

Keluaran (output)







Tabel 3.3 Input/Output Test Case 3

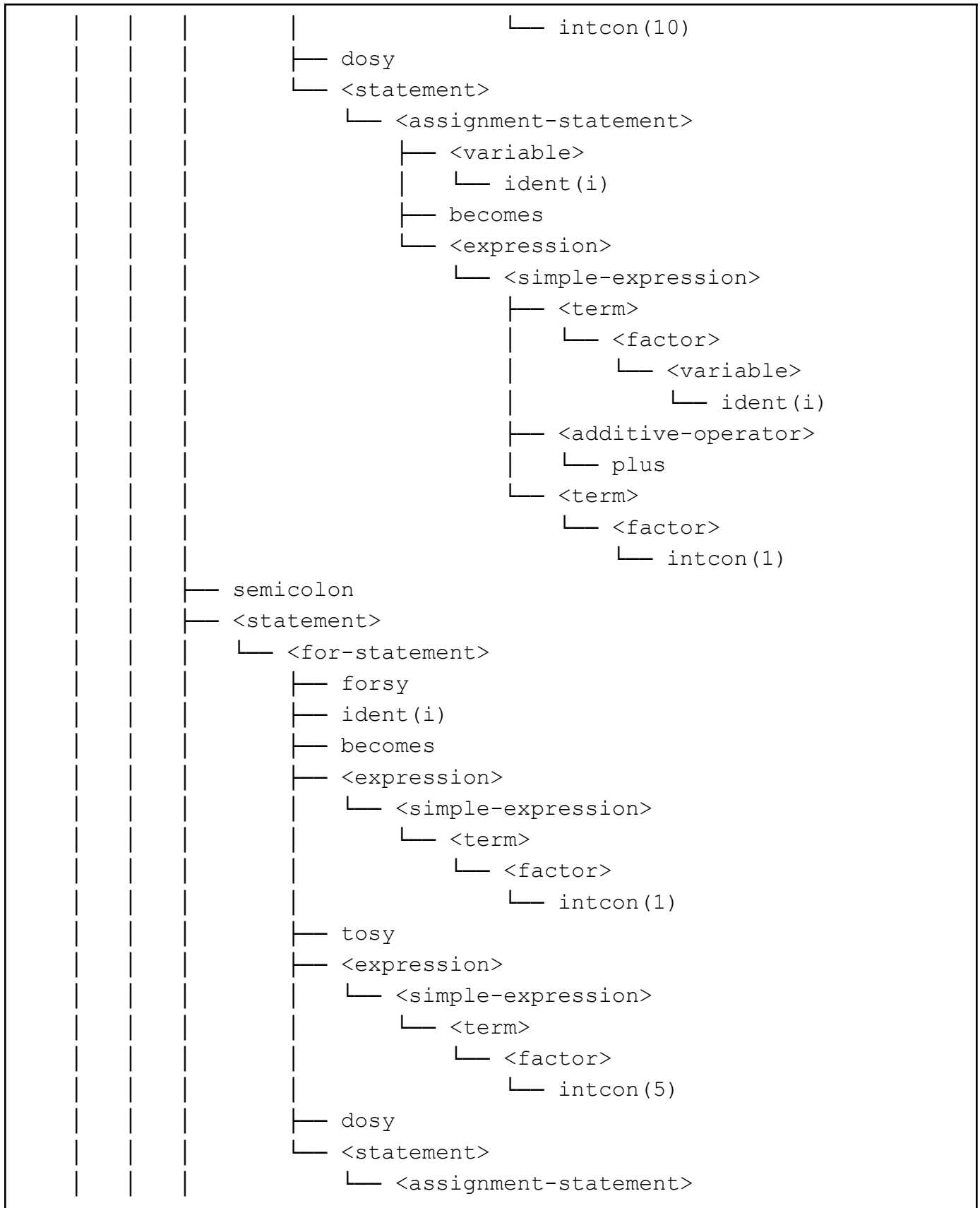
3.4 Test Case 4

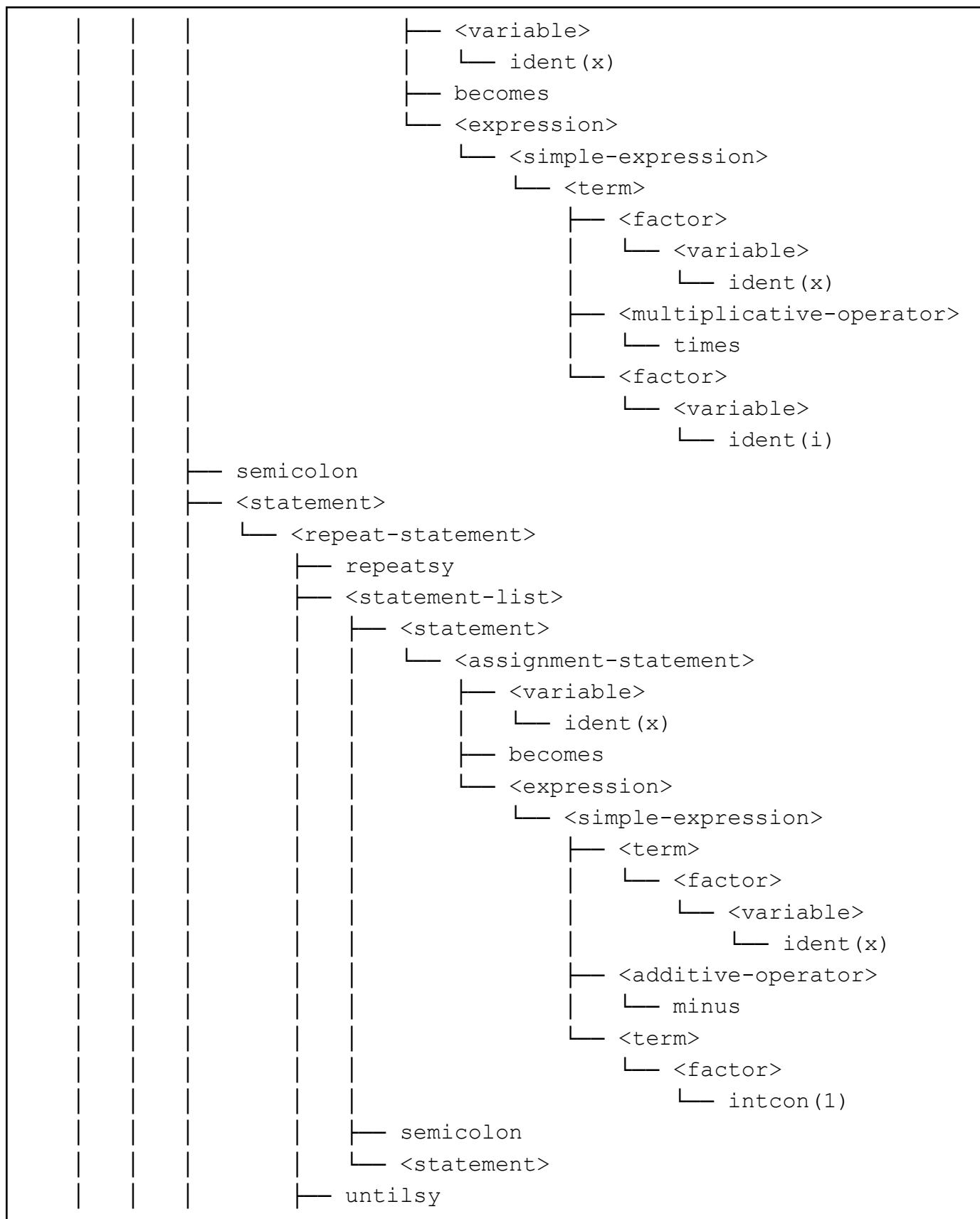
Masukan (input)
<pre>program ControlFlow; var i, x: integer; begin while i < 10 do i := i + 1; for i := 1 to 5 do x := x * i; repeat x := x - 1;</pre>

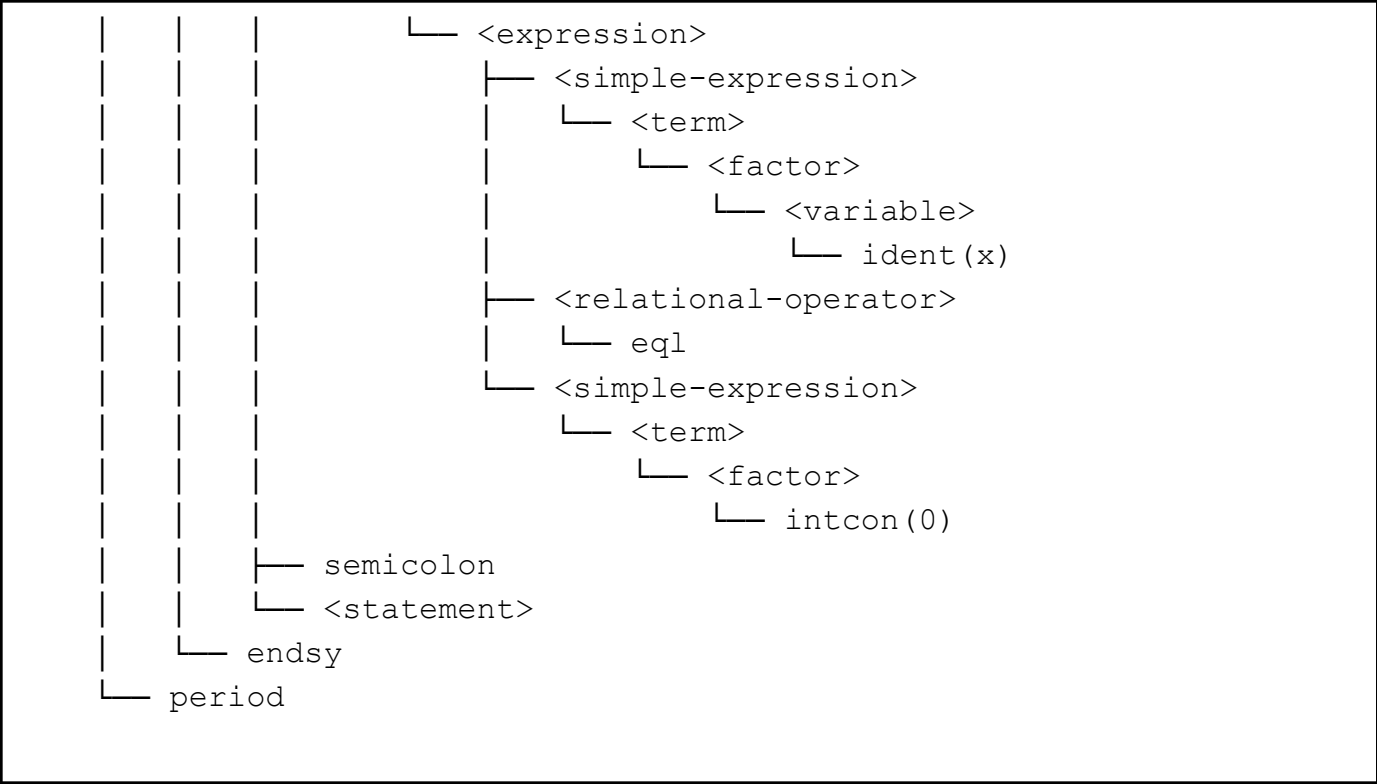
```
until x == 0;
end.
```

Keluaran (output)

```
└─ <program>
  └─ <program-header>
    └─ programsy
    └─ ident(ControlFlow)
    └─ semicolon
  └─ <declaration-part>
    └─ <var-declaration>
      └─ varsy
      └─ <identifier-list>
        └─ ident(i)
        └─ comma
        └─ ident(x)
      └─ colon
      └─ <type>
        └─ ident(integer)
      └─ semicolon
  └─ <compound-statement>
    └─ beginsy
    └─ <statement-list>
      └─ <statement>
        └─ <while-statement>
          └─ whilesy
          └─ <expression>
            └─ <simple-expression>
              └─ <term>
                └─ <factor>
                  └─ <variable>
                    └─ ident(i)
            └─ <relational-operator>
              └─ lss
            └─ <simple-expression>
              └─ <term>
                └─ <factor>
```







Tabel 3.4 Input/Output Test Case 4

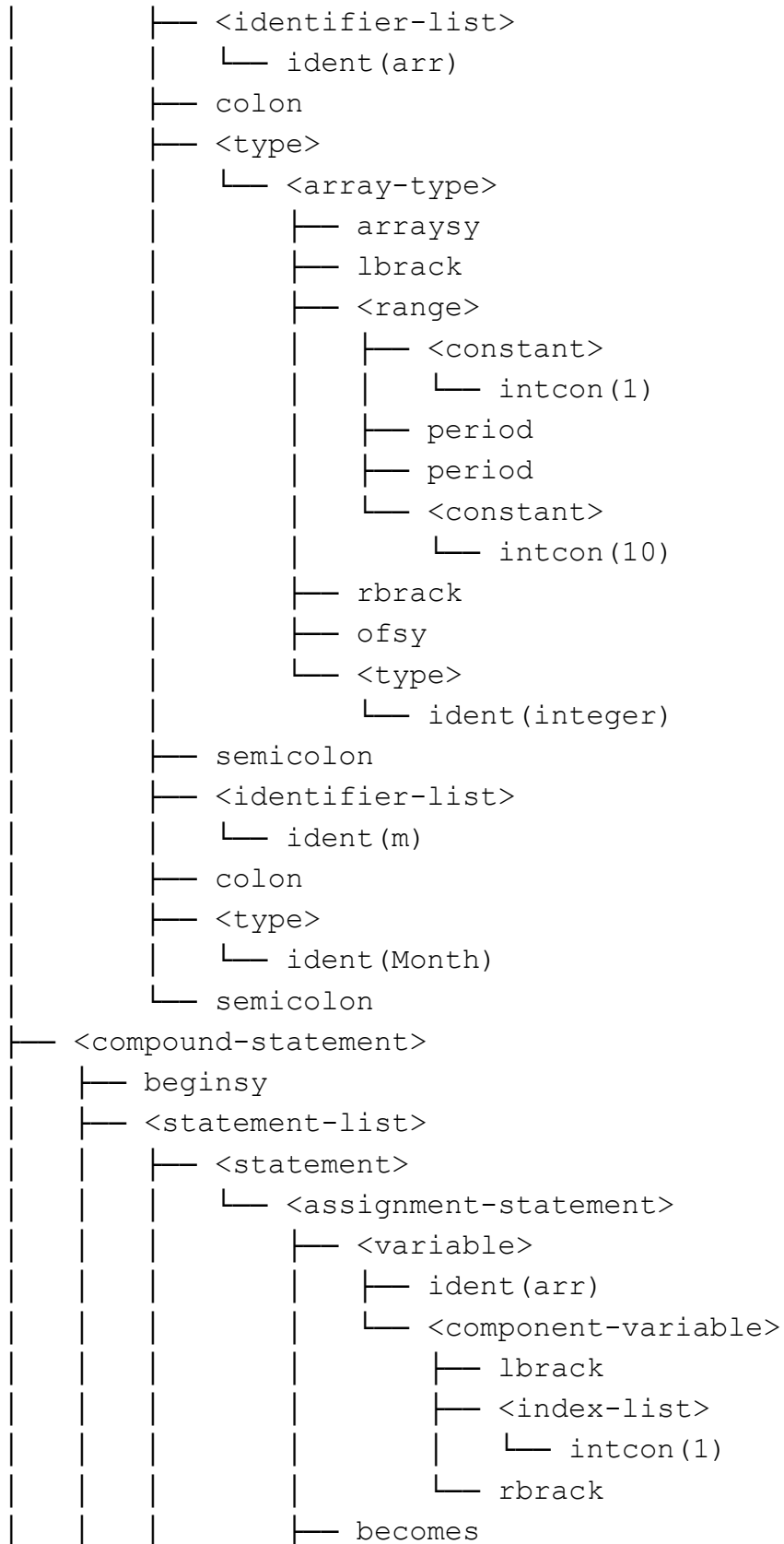
3.5 Test Case 5

Masukan (input)
<pre>program TypeDecl; type Range == 1 .. 10; Month == (Jan, Feb, Mar, Apr, May, Jun); var arr: array[1 .. 10] of integer; m: Month; begin arr[1] := 100; arr[2] := arr[1] + 50 end.</pre>
Keluaran (output)
<pre>└─ <program></pre>

```

├─ <program-header>
│   ├── programsy
│   ├── ident (TypeDecl)
│   └── semicolon
├─ <declaration-part>
│   ├── <type-declaration>
│   │   ├── typesy
│   │   ├── ident (Range)
│   │   ├── eql
│   │   ├── <type>
│   │   │   └── <range>
│   │   │       ├── <constant>
│   │   │       │   └── intcon (1)
│   │   │       ├── period
│   │   │       ├── period
│   │   │       └── <constant>
│   │   │           └── intcon (10)
│   │   ├── semicolon
│   │   ├── ident (Month)
│   │   ├── eql
│   │   ├── <type>
│   │   │   └── <enumerated>
│   │   │       ├── lparent
│   │   │       ├── ident (Jan)
│   │   │       ├── comma
│   │   │       ├── ident (Feb)
│   │   │       ├── comma
│   │   │       ├── ident (Mar)
│   │   │       ├── comma
│   │   │       ├── ident (Apr)
│   │   │       ├── comma
│   │   │       ├── ident (May)
│   │   │       ├── comma
│   │   │       ├── ident (Jun)
│   │   │       └── rparent
│   │   └── semicolon
│   └── <var-declaration>
│       ├── varsy

```




```

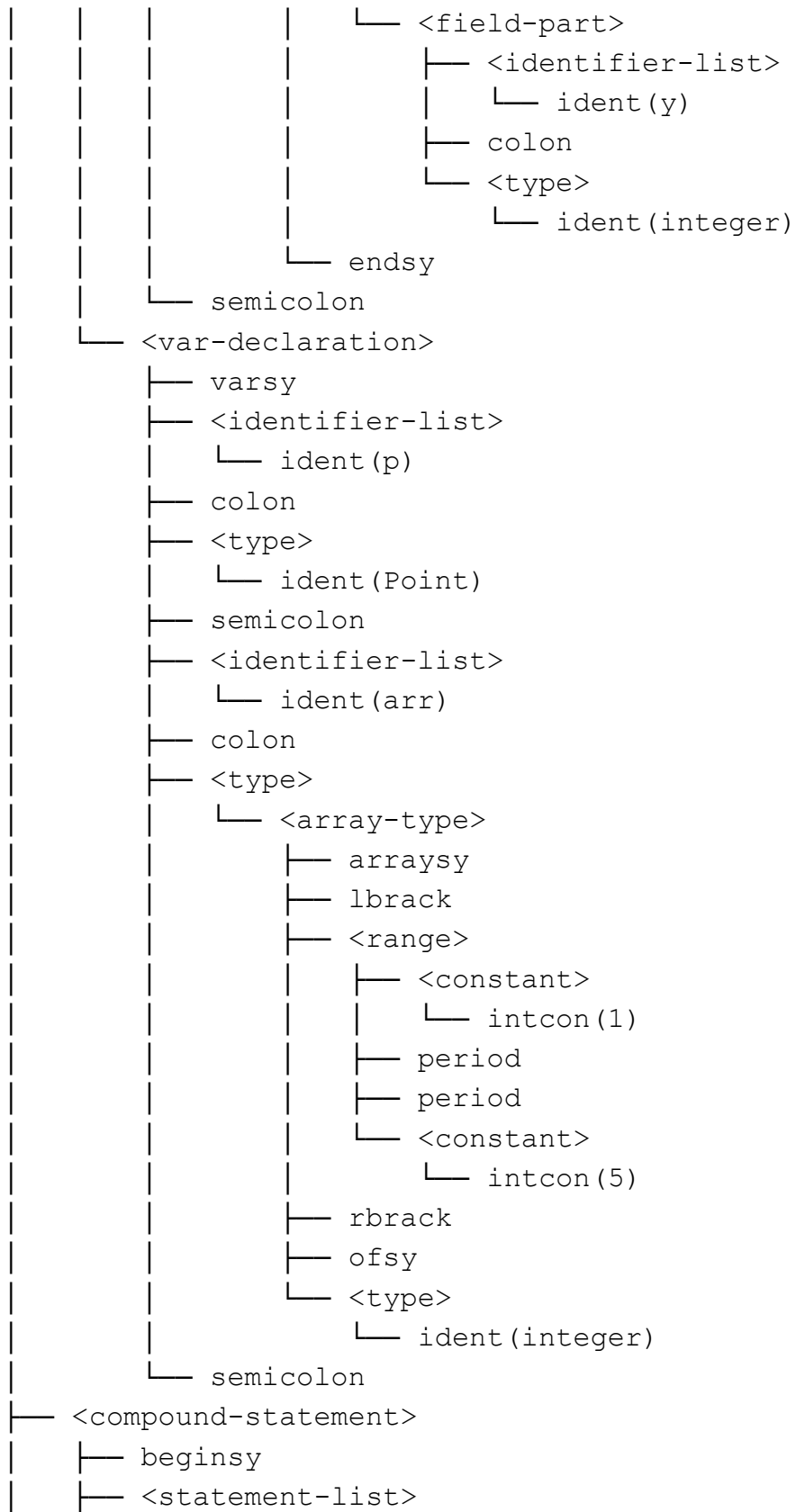
└─ <expression>
  └─ <simple-expression>
    └─ <term>
      └─ <factor>
        └─ intcon(100)
└─ semicolon
└─ <statement>
  └─ <assignment-statement>
    └─ <variable>
      └─ ident(arr)
      └─ <component-variable>
        └─ lbrack
        └─ <index-list>
          └─ intcon(2)
        └─ rbrack
    └─ becomes
    └─ <expression>
      └─ <simple-expression>
        └─ <term>
          └─ <factor>
            └─ <variable>
              └─ ident(arr)
              └─ <component-variable>
                └─ lbrack
                └─ <index-list>
                  └─ intcon(1)
                └─ rbrack
          └─ <additive-operator>
            └─ plus
          └─ <term>
            └─ <factor>
              └─ intcon(50)
└─ endsy
└─ period

```

Tabel 3.5 Input/Output Test Case 5

3.6 Test Case 6

Masukan (input)
<pre> program TypeDecl; type Range == 1 .. 10; Month == (Jan, Feb, Mar, Apr, May, Jun); var arr: array[1 .. 10] of integer; m: Month; begin arr[1] := 100; arr[2] := arr[1] + 50 end. </pre>
Keluaran (output)
<pre> └─ <program> └─ <program-header> └─ programsy └─ ident(RecordTest) └─ semicolon └─ <declaration-part> └─ <type-declaration> └─ typesy └─ ident(Point) └─ eql └─ <type> └─ <record-type> └─ recordsy └─ <field-list> └─ <field-part> └─ <identifier-list> └─ ident(x) └─ colon └─ <type> └─ ident(integer) └─ semicolon └─ semicolon └─ semicolon └─ semicolon └─ semicolon </pre>




```

└─ plus
└─ <term>
  └─ <factor>
    └─ intcon(20)
└─ semicolon
└─ <statement>
  └─ <assignment-statement>
    └─ <variable>
      └─ ident(arr)
        └─ <component-variable>
          └─ lbrack
            └─ <index-list>
              └─ intcon(3)
            └─ rbrack
    └─ becomes
    └─ <expression>
      └─ <simple-expression>
        └─ <term>
          └─ <factor>
            └─ <variable>
              └─ ident(arr)
                └─ <component-variable>
                  └─ lbrack
                    └─ <index-list>
                      └─ intcon(1)
                    └─ rbrack
        └─ <additive-operator>
          └─ plus
        └─ <term>
          └─ <factor>
            └─ <variable>
              └─ ident(arr)
                └─ <component-variable>
                  └─ lbrack
                    └─ <index-list>
                      └─ intcon(2)
                    └─ rbrack

```

endsy

└ period

Tabel 3.6 Input/Output Test Case 6

3.7 Test Case 7

Masukan (input)
<pre>program TypeTest; const MAX == 10; var n: integer; type Season == (Spring, Summer, Fall, Winter); procedure printNum(n: integer); begin writeln(n); end; begin n := MAX; printNum(n); end.</pre>
Keluaran (output)
<pre>Syntax error at line 6: declaration order violation - 'typesy' appears out of order (required order: const, type, var, subprogram)</pre>

Tabel 3.7 Input/Output Test Case 7

3.8 Test Case 8

Masukan (input)
<pre>program ErrorTest; var</pre>

<pre> x: integer; begin x := 5 x := x + 1; end.</pre>
Keluaran (output)
Syntax error at line 6 col 3: unexpected token 'ident(x)', expected 'endsy'

Tabel 3.8 Input/Output Test Case 8

3.9 Test Case 9

Masukan (input)
<pre> program ErrorTest; var x: integer; begin x := * 5 end.</pre>
Keluaran (output)
Syntax error at line 4 col 8: unexpected token 'times' in expression

Tabel 3.9 Input/Output Test Case 9

3.10 Test Case 10

Masukan (input)
<pre> program ErrorTest; begin x := 5; y := 10 .</pre>

Keluaran (output)
Syntax error at line 5 col 1: unexpected token 'period', expected 'endsy'

Tabel 3.10 Input/Output Test Case 10

3.11 Test Case 11

Masukan (input)
<pre> program ErrorTest; procedure foo(x: integer;); begin end; begin end. </pre>
Keluaran (output)
<pre> Syntax error at line 2: unexpected semicolon in formal-parameter-list (trailing semicolon not allowed before ')') </pre>

Tabel 3.11 Input/Output Test Case 11

3.12 Test Case 12

Masukan (input)
<pre> program ErrorTest; var i: integer; begin for i := 1 5 do i := i + 1 end. </pre>
Keluaran (output)
<pre> Syntax error at line 4: expected 'to' or 'downto' in </pre>


```
for-statement, got 'intcon'
```

Tabel 3.12 Input/Output Test Case 12

BAB IV

KESIMPULAN DAN SARAN

4.1 Kesimpulan

Pada Milestone 2 ini, diperlukan implementasi tahap kedua dari Arion Compiler, yaitu Syntax Analysis (Parser) menggunakan algoritma Recursive Descent. Parser menerima daftar token hasil Lexical Analysis dari Milestone 1 dan membangun Parse Tree yang merepresentasikan struktur hierarkis program sesuai grammar bahasa Arion.

Parser telah berhasil diimplementasikan menggunakan algoritma Recursive Descent, di mana setiap production rule grammar direpresentasikan sebagai satu fungsi tersendiri. Pendekatan ini menghasilkan kode yang modular, mudah dipahami, dan alur eksekusinya mengikuti struktur grammar secara langsung. Grammar bahasa Arion berhasil di-hardcode secara lengkap dan terstruktur, mencakup seluruh node yang didefinisikan dalam spesifikasi, mulai dari struktur program utama, deklarasi konstanta, tipe, variabel, subprogram, berbagai jenis statement, hingga ekspresi dengan hierarki operator yang benar.

Parse Tree yang dihasilkan telah sesuai dengan spesifikasi, di mana setiap node merepresentasikan tepat satu production rule grammar. Struktur tree dicetak ke terminal dengan format visual yang jelas menggunakan karakter box-drawing dan disimpan ke dalam file .txt. Selain itu, error handling berhasil diimplementasikan dengan pesan yang informatif dengan menampilkan nomor baris dan keterangan token yang tidak sesuai sehingga dapat membantu pengguna mengidentifikasi letak kesalahan sintaks pada program. Program tidak crash saat menemukan error dan tetap menampilkan pesan yang dapat dipahami pengguna.

4.2 Saran

Berdasarkan dari pengalaman mengerjakan Milestone 2, terdapat beberapa saran yang dapat menjadi pertimbangan untuk pengembangan di milestone-milestone selanjutnya, antara lain:

1. Perbaiki Lexer untuk field access. Saat ini lexer tidak dapat menangani akses field record seperti `p.x` tanpa spasi karena DFA membacanya sebagai `ident(p) + unknown(.x)`. Perlu ditambahkan penanganan khusus di state setelah IDENT untuk membedakan antara period sebagai pemisah field access dan period sebagai akhir program.
2. Error recovery. Saat ini parser berhenti pada error pertama yang ditemukan. Pada milestone selanjutnya, sebaiknya ditambahkan mekanisme error recovery seperti panic mode agar parser dapat melanjutkan analisis setelah menemukan error dan melaporkan semua error sekaligus dalam satu kali eksekusi.
3. Validasi token UNKNOWN dari Lexer. Saat ini jika lexer menghasilkan token UNKNOWN, parser akan langsung melempar syntax error. Sebaiknya ditambahkan pengecekan khusus di awal parser untuk mendeteksi dan melaporkan token UNKNOWN dari lexer secara terpisah agar pesan errornya lebih jelas.
4. Persiapan untuk Semantic Analysis. Parse Tree yang dihasilkan saat ini masih berupa tree lengkap dengan semua node terminal dan non-terminal. Untuk mempersiapkan tahap Semantic Analysis pada Milestone 3, perlu dipertimbangkan struktur data tambahan seperti symbol table yang dapat dibangun bersamaan saat traversal Parse Tree.

BAB V

ATURAN GRAMMAR

1. `<program>` → `<program-header>` `<declaration-part>`
`<compound-statement>` period
2. `<program-header>` → `programsy ident semicolon`
3. `<declaration-part>` → `<const-declaration>*`
`<var-declaration>*` `<type-declaration>*`
`<subprogram-declaration>*`
4. `<const-declaration>` → `constsy (ident eql constant semicolon)+`
5. `<constant>` → `charcon | string | [(plus | minus)? (ident | intcon | realcon)]`
6. `<type-declaration>` → `typesy (ident eql type semicolon)+`
7. `<var-declaration>` → `varsy (<identifier-list> colon <type> semicolon)+`
8. `<identifier-list>` → `ident (comma ident)*`
9. `<type-declaration>` → `typesy (ident eql <type> semicolon)+`
10. `<type>` → `ident | <array-type> | <range> | <enumerated> | <record-type>`
11. `<array-type>` → `arraysy lbrack (<range> | ident) rbrack ofsy <type>`
12. `<range>` → `<constant>` period period `<constant>`
13. `<enumerated>` → `lparent ident (comma ident)* rparent`
14. `<record-type>` → `recordsy <field-list> endsy`
15. `<field-list>` → `<field-part> (semicolon <field-part>)*`
16. `<field-part>` → `<identifier-list> colon <type>`
17. `<subprogram-declaration>` → `<procedure-declaration> | <function-declaration>`
18. `<procedure-declaration>` → `proceduresy ident (<formal-parameter-list>)? semicolon <block> semicolon`
19. `<function-declaration>` → `functionsy ident (<formal-parameter-list> )? colon ident semicolon <block> semicolon`
20. `<block>` → `<declaration-part>` `<compound-statement>`
21. `<formal-parameter-list>` → `lparent <parameter-group> (semicolon <parameter-group>)* rparent`

22. <parameter-group> → <identifier-list> colon (ident | <array-type>)
23. <compound-statement> → beginsy <statement-list> endsy
24. <statement-list> → <statement> (semicolon <statement>)*
25. <statement> → <assignment-statement> | <if-statement> | <case-statement> | <while-statement> | <repeat-statement> | <for-statement> | <procedure/function-call> | ε
26. <variable> → ident <component-variable>*
27. <component-variable> → (lbrack <index-list> rbrack) | (period ident)
28. <index-list> → (intcon | charcon | ident) (comma <index-list>)*
29. <assignment-statement> → <variable> becomes <expression>
30. <if-statement> → ifsy <expression> thensy <statement> (elsesy <statement>)?
31. <case-statement> → casesy <expression> ofsy <case-block> endsy
32. <case-block> → <constant> (comma <constant>)* colon <statement> (semicolon <case-block>?)*
33. <while-statement> → whilesy <expression> dosy <statement>
34. <repeat-statement> → repeatsy <statement-list> untilsy <expression>
35. <for-statement> → forsy ident becomes <expression> (tosy | downtosy) <expression> dosy <statement>
36. <procedure/function-call> → ident (lparent <parameter-list>? rparent)?
37. <parameter-list> → <expression> (comma <expression>)*
38. <expression> → <simple-expression> (<relational-operator> <simple-expression>)?
39. <simple-expression> → (plus | minus)? <term> (<additive-operator> <term>)*
40. <term> → <factor> (<multiplicative-operator> <factor>)*
41. <factor> → ident | intcon | realcon | charcon | string | lparent <expression> rparent | notsy <factor> | <procedure/function-call> | <variable>
42. <relational-operator> → eql | neq | gtr | geq | lss | leq
43. <additive-operator> → plus | minus | orsy

44. <multiplicative-operator> → times | rdiv | idiv | imod
| andsy

LAMPIRAN

Link Repository : <https://github.com/gabriellaalubis/OTH-Tubes-IF2224-2026>

Pembagian tugas :

Nama	NIM	Pembagian	Persentase
Gabriella Botimada Lubis	13524006	● Implementasi Kode Parser ● Testing ● Bug Fixing ● Penyusunan Laporan	33.3%
Stefani Angeline Oroh	13524064	● Implementasi Kode Parser ● Testing ● Bug Fixing ● Penyusunan Laporan	33.3%
Reva Natania Sitohang	13524098	● Implementasi Kode Parser ● Testing ● Bug Fixing ● Penyusunan Laporan	33.3%